Different Clustering Methods and Comparison of their Performances in NCI60 Dataset

Ahan Chakraborty

April 14, 2026

1 Introduction

Clustering is an unsupervised learning method in which we divide the dataset into different sub-groups or clusters where the clusters are homogeneous within themselves but are heterogeneous when compared between themselves. In this project report we describe various classical clustering techniques and their modification for High Dimensional Low Sample (HDLS) datasets and we study their performances on such a dataset NCI60.

2 Traditional Clustering Techniques

2.1 K-means Clustering

The most popular method of cluster is arguably $K-$means clustering. Fix a number K which denotes the number of clusters in which the given dataset has to be divided in.

1. Randomly assign a number, from 1 to K, to each of the observations. These serve as initial cluster assignments for the observations.
2. Iterate until the cluster assignments stop changing:
   a) For each cluster , compute the cluster centroid, The k th cluster centroid is the vector of the p feature means for the observation in the k th cluster.
   b) Assign each observation to the cluster whose centroid is closest (where closest is defined using Euclidean distance)

As we are starting with random initial configurations, it is advised to do the process multiple times and choose the clustering that has minimal within-cluster-sum of squares of distances.

2.2 Hierarchical CLustering

A potential problem for k-means clustering is that we have to specify the parameter k. In Hierarchical clustering, after constructing a single structure called dendogram we may construct clusterings with different number of clusters by simply cutting the dendogram at different depth. We discuss the traditional bottom-up hierarchical clustering in this section.

1. We begin with N observations and a measure of dissimilarity such as Euclidean norm. First treat all the observations as its own cluster.
2. For $i = n, n-1, ..., 2$:
   a) Examine all pairwise inter-cluster dissimilarities among the i clusters and identify the pair of clusters that are least dissimilar. Fuse them. The dissimilarity between these two clusters indicates the height in the dendo-gram at which the fusion should be placed.
   b) Compute the new pairwise inter-cluster dissimilarities among the $i-1$ remaining clusters.

Now note that the notion of dissimilarity between two individual is very easy but what about the notion of dissimilarity between two clusters? For that we use a concept of linkages. The popular linkages are as following:

1. Complete linkage: The maximal dissimilarity between an observation in cluster A and an observation in cluster B is taken to be dissimilarity between the two clusters.
2. Single linkage: The minimal dissimilarity between an observation in cluster A and an observation in cluster B is taken to be dissimilarity between the two clusters.
3. Average linkage: The average dissimilarity between an observation in cluster A and an observation in cluster B is taken to be dissimilarity between the two clusters.

2.3 Convex Clustering (Chi-Molstad-Gao-Chi et al.)

Given n data pts $X = \{x_1, \ldots, x_n\} \subseteq \mathbb{R}^p$, we seek cluster centroids $u_i \in \mathbb{R}^p$ for each x_i by minimizing the convex criterion as follows:

$$\min_{u \in \mathbb{R}^{np}} E_\gamma(u) = \frac{1}{2} \sum_{i=1}^{n} \|x_i - u_i\|_2^2 + \gamma \sum_{(i,j) \in \mathcal{E}} w_{ij} \|u_i - u_j\|_2$$

Where γ is the **tuning parameter** the second term is the penalty for the model complexity and $w_{ij} \geq 0$ are weights denoting similarity i th and jth observation. u is the long vector we would get if we stack the cenntroid vectors on top of one another. And, $\mathcal{E} = \{(i,j) \in [n]^2 | w_{ij} > 0\}$
Remark: Note that for $E_\gamma(u)$, the first term is "strongly convex" and the second term is convex making the objective function strongly convex. And hence for every fixed γ, there is a unique minimizer $u(\gamma)$ and hence unique centroids $u_i(\gamma)$ for all i's.

Clustering Centroids: The Role of γ

Why bother about γ?

- When $\gamma = 0$: $u_i = x_i$. Each point is getting its own cluster.
- As γ increases from 0: Clusters fuse together. γ trades off the emphasis between data fit and differences between pairs of centroids.
- When γ is very large: u_i is the center of mass (COM) of X.

Let $\mathcal{P} = \{P_1, \ldots, P_k\}$ denote a partition of X inducing index partition $\mathcal{I}$.

Weights

$w_{ij} \to$ intuitively measuring similarities between x_i and x_j. If i, j are "close", then their centroids should not be far, so a greater penalty should be imposed if the centroids of the clusters they belong to are far apart.

Weights in Practice

- **Inverse Euclidean Norm:** $w_{ij} = \frac{1}{\|x_i - x_j\|_2}$
- **Gaussian Kernel:** $w_{ij} = \exp\left(-\frac{\|x_i - x_j\|_2}{\sigma_{ij}}\right)$, where $\sigma_{ij} = \sigma_i \cdot \sigma_j$.

σ_i is a local measure of dispersion around x_i, e.g., the median Euclidean distance between x_i and its nearest neighbors.

Example

The following example shows how the weights and the tuning parameter affect the choice of the centroids.

Let us consider this figure having 5 clusters having 5 observations each, each observation within the same cluster is denoted by same symbol and moreover there are two super-clusters represented by same color. The weights can take values between $w_1 \geq w_2 \geq w_3$. One of the many ways of constructing weights for a pair i,j is as follows: Consider a graph G with vertex set $[n]$. Construct an edge between i and j if X_i is the set of k nearest neighbours of X_j (We assume symmetry in this relation here for simplicity). We will get a graph which reflects the geometry of the dataset on hand. Now if it has some connected components inducing index partition $\{\mathcal{I}_1, ..., \mathcal{I}_k\}$, and partition $\{\mathcal{E}_1, ..\mathcal{E}_k\}$ of the dataset, now if i, j belong to same $\mathcal{I}_k$ we call the weight w_{ij} to be inter-cluster weight and otherwise intra-cluster weight. The following figure how the unifrmity of intra-cluster weights, intra-supercluster weights and inter-supercluster weights affect the choice of the centroids as γ increases. For example, if all the weights of uniform , then as no super-cluster centroid has extra preference, hence we will see the centroids to move from individual datapoints to the global centroid as tuning para. increases. And if $w_1 >> w_2 >> w_3$, we will see the centroids first at individual points, then at each centroid of the super-clusters and finally in the global centroid as γ increases eventually.

3 Curse of Dimensionality : Problems in High Dimensional Low Sample (HDLS) Data

As the number of features increases, we say the dimensionality increases. And we can show mathematically, that in high dimension the volume "escapes" to the corners. If we consider the unit hyper-sphere touching the faces of the unit hyper-cube we can show that the ratio of the volume of the unit hyper-sphere to that of the volume of the hyper-cube tends to 0 as dimensionality goes to ∞. Hence, as the dimensionality increases, a larger percentage of the training data resides near the corners of the feature space. And points closer to the centroid tends to be closer to many datapoints. In high dimensions, the difference between the distance to the nearest neighbor and the farthest neighbor effectively disappears. And so, distance based methods tend to lose their usefulness for High Dimensional datas.

4 Adjustments for CoD

4.1 Sparse Hierarchical Clustering (Hastie-Tibshirani et al.)

One way to overcome this is by taking weights. Now in the HD datasets, many features might be noise. So, we may overcome curse of dimensionality by introducing sparsity in the weight vector, i.e. making many components zero.

Sparse Hi. Clustering : (Hastie, Tibshirani, SLS).

$$D_{i,i'} = \sum_{j=1}^{p} d_{i,i',j}$$

$$d_{i,i',j} = (x_{ij} - x_{i'j})^2$$

Now instead of considering $D_{i,i'}$ we take a weighted euclidean norm such that

$$\tilde{D}_{i,i'} = \sum_{j=1}^{p} w_j d_{i,i',j}$$

where the weight w_j will reflect some notion of importance of the variable j. We will want to introduce sparsity which will make many of the co-ordinates of w to be 0 and will do some sort of feature selection automatically.

Let Δ be the $N^2 \times p$ matrix where Δ_{ij} is the pairwise distance $d_{i,i',j}$ $\therefore$ $\Delta\mathbf{1}$ denotes the vectorized version of $D_{i,i'}$ and Δw denotes the same of $\tilde{D}_{i,i'}$

Let Δ be a $N^2 \times p$ matrix with Δ_j denotes the pairwise sq. differences of the jth co-ordinates of the observations. Note that $\Delta \mathbf{1}$ is the vectorized form of the euclidean norms and Δw is the vectorized form of the weighted euclidean distances. Now for any fixed value of w, we want to capture the maximum dissimilarity we can extract from the vector Δw. So we want to maximize the quantity

$$u^T \Delta w$$

But what are the constraints? restricting for a $u^T \Delta w$ to be arbitrarily large due to scaling of u & w, we introduce the constraints $||u||_2 \leq 1$ & $||w||_2 \leq 1$. Then we introduce $w \geq 0$ for the weights to be non-negative. and also $||w||_1 \leq s$ where s is some tuning parameter.The rationale behind taking l_1 norm here is that it introduces sparsity to the weight vector. For example, consider 2 dimensions, the region $||w||_1 \leq s$ defines a square (diamond).

The optimal weight solution is very likely to be one of these vertices (as convex problem) and as on the axis, it will get many co-ordinates equal to 0. This is motivated from the Lasso regression(Tibshirani et al.)

4.2 Sparse Convex Clustering (Wang-Zhang-Sun-Fang et al.)

The Sparse Convex Clustering method solves an optimization problem which is a modification of the convex clustering optimization problem,

$$\min_{u \in \mathbb{R}^{np}} E_\gamma(u) = \frac{1}{2} \sum_{i=1}^{n} \|x_i - u_i\|_2^2 + \gamma_1 \sum_{(i,j) \in \mathcal{E}} w_{ij} \|u_i - u_j\|_2 + \gamma_2 \sum_{j=1}^{p} v_j ||a_j||_2$$

where a_j denotes the j th column of the matrix $U = [u_1 : u_2 : ...u_n]^T$ (The matrix is constructed by stacking the centroid vectors as rows and its j th column is the vector with entries as the j th co-ordinates of the centrd vectors, so we are penalizing over the size of the co-ordinates of the centroid vectors by the second penalty term) tuning parameter γ_1 controls the cluster size and tuning parameter γ_2 controls the number of informative features. In the group-lasso penalty, the weight v_j plays an important role to adaptively penalize the features.

In the objective function of sparse convex clustering, it can be shown that the second group-lasso-type penalty enforces the global sparsity condition; that is, the elements of each centroid vector a_j would be all zero or all nonzero. Such penalty is considered for the feature selection purpose. This global sparsity condition can be relaxed in two directions. First, we can replace the second penalty, , by a lasso type of penalty, $\gamma_2 \sum_{j=1}^{p} v_j ||a_j||_2$ by a lasso-penalty term $\gamma_2 \sum_{j=1}^{p} v_j ||a_j||_1$. Second, we can also add another penalty, $\gamma_3 \sum_{j=1}^{p} t_j ||a_j||_1$, to the objective function, which is the so-called sparse-group-lasso penalty (Friedman et al., 2010).

4.3 Biconvex Clustering (Chakraborty- Xu et al)

Chakraborty-Xu et al. gives yet another modification to the objective function of Convex Clustering to adjust for high dimensionality is as follows :

$$\min_{u \in \mathbb{R}^{np}} E_{\gamma,\lambda}(u, v) = \sum_{i=1}^{n} \|x_i - u_i\|_v^2 + \gamma \sum_{(i,j) \in \mathcal{E}} w_{ij} \|u_i - u_j\|_2$$

subject to $v \succeq 0, ||v||_1 = 1$

Where the norm induced by a vector v is defined as follows:

$$||y||_w = \sum_{i=1}^{p} (w_i^2 + \lambda w_i) v_i^2$$

We see that the first component $\sum_{i=1}^{n} \|x_i - u_i\|_v^2$ assesses the fit between the centroids u and the data X measured in a way that is weighed by u. Note that if we fix all $v_l \propto 1$ throughout, $||y||_v$ becomes a scalar multiple of the Euclidean norm $||y||_2$, and thus minimizing this objective function is equivalent to convex clustering as a special case. It can be also shown that the objective function is biconvex in (u, v). Under the simplex constraint $||v||_1 = 1$, the λv_l term promotes sparsity (Witten and Tibshirani, 2010). Thus the objective in v entails feature weighing as well as selection.

4.4 MADD(Ghosh-Sarkar et al.)

Ghosh- Sarkar et al. gives an alternative data-driven dissimilarity measure for clustering called **M**ean of **A**bsolute **D**ifferences of pairwise **D**istances (MADD) given by :

$$\rho_0(u,v) = \frac{1}{n-2} \sum_{z \in \mathcal{X} \setminus \{u,v\}} \quad | \quad ||u-z||_2 - ||v-z||_2 \quad |$$

It can be shown that if U, V comes from same population then as $d \to \infty$, we will have $\rho_0(U,V) \xrightarrow{P} 0$, otherwise it will converge to a constant in probility. So, in high dimension, MADD will preserve the neighbourhood structure of the dataset with high probability and hence MADD based clustering methods can be expected to perform better than othe euclidean distance based clustering techniques. Under certain assumptions,MADD distance have many statistical properties for large dimensionality. The proofs are beyond the scope of the project and hence are skipped.

5 Rand Index

Rand Index is a tool to measure the performance of a clustering algoritm δ. It is defined as:

$$R(\delta) = \frac{1}{\binom{n}{2}} \sum_{i<j} \mathbb{1}\left[\mathbb{1}\{\delta(X_i) - \delta(X_j)\} + \mathbb{1}\{\mathcal{C}(X_i) - \mathcal{C}(X_j)\} = 1\right]$$

Essentially we are checking in how many pairs clustering algorithm has kept 2 observations in the same group where they were in 2 different groups originally or the other way round. Larger value of Rand index indicates poor performance of the clustering algorithm.

6 Simulations

6.1 Dataset

We use the NCI60 cancer cell line micro-array here, avaialable in Kaggle, also in the book *Introduction to Statistical Learning* by Hastie-Witten-Tibshirani. The data consists of 6830 gene expression measurements on 64 cancer cell lines.

For validation and comparison in the performance of the clustering algorithms, we use the labelings provided in the book mentioned as our ground-truth. According to the labeling the 14 found types of cancer are as following : CNS, RENAL, BREAST, NSCLC, UNKNOWN, OVARIAN, MELANOMA, PROSTATE, LEUKEMIA, K562B-repro, K562A-repro, COLON, MCF7A-repro, MCF7D-repro.

We denote them with the numbers 1 to 14. The number of observations in each group are the following : 5 instances of CNS(1) , 9 observations of Renal(2) , 7 observations labeled Breast(3) , 9 NSCLC(4) , 6 Ovarian instances (6), 8 Melanomas(7), 2 Prostate instances(8), 6 Lukemia observations (9), 7 Colon Cancer instances (12) , 1 each of K562B-repro(10), K562A-repro(11), MCF7A-repro(13), MCF7D-repro(14) and 1 unknown(5).

6.2 R code

```
#1. install packages
#install.packages("ISLR2")
#install.packages("devtools")
#install.packages("sparcl")
#install.packages("HDLSSkST")
#library(devtools)
#install.packages("factoextra")
#install.packages("Rtsne")
#devtools::install_github("DataSlingers/clustRviz")
#--------------------
#2. Load Packages
library(ISLR2)
library(clustRviz)
library(sparcl)
library(HDLSSkST)
library(Rtsne)
```

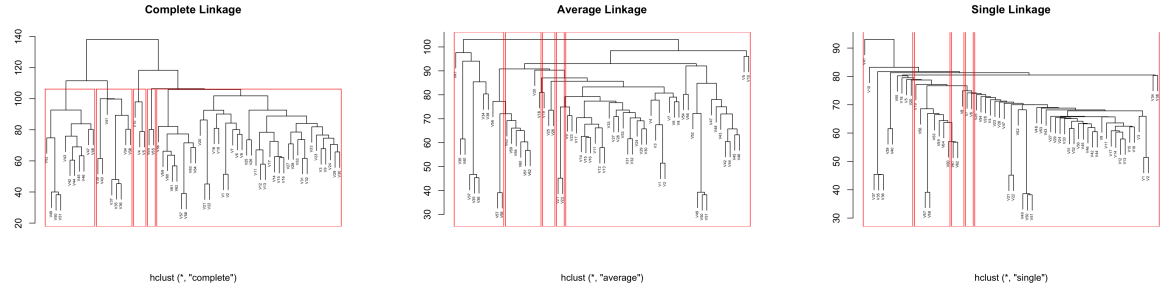
```r
#----------------------
#3. Load the Dataset
cancer.data =NCI60$data
cancer.lab = NCI60$labs
#cancer.data = cancer.data[,-1]
#head(cancer.data)
#View(cancer.data)
dim(cancer.data)
#-----------------
#4. Scale the dat afor PCA(optional)
cancer.data2 = scale(cancer.data)
#----------------------------
#5. Rand Index
Rand = function(X, Y) {
  n = length(X)
  r = 0
  for(i in 1:(n-1)) {
    for (j in (i+1):n) {
      if (((X[i] == X[j]) & (Y[i] != Y[j])) | ((X[i] != X[j]) & (Y[i] == Y[j]))) {
        r = r + 1
      }
    }
  }
  return(r / choose(n, 2))
}
#6. Fix the Number of clusters
k = 5
#7. Do pca
#color coding
unique(cancer.lab)
colors = numeric(length(unique(cancer.lab)))
for ( i in 1:64)

{ colors[i] = which(unique(cancer.lab) == cancer.lab[i])}

pc.cancer = prcomp(cancer.data)

#8.Hierarchical Clustering
#complete linkage
hcluster.com = hclust( dist(cancer.data), method = "complete")
#dev.new()

plot( hcluster.com, xlab = "", ylab = "", main = "Complete Linkage", cex = 0.4)
hc.com = cutree(hcluster.com , k )
rect.hclust(hcluster.com, k = k, border = "red")
#table(hc.com, cancer.lab)
Rand(hc.com,cancer.lab)
#--------------------------
#average linkage
hcluster.avg = hclust (dist(cancer.data), method  = "average")
#dev.new()
plot( hcluster.avg, xlab = "", ylab = "", main = "Average Linkage", cex  = 0.4)

rect.hclust(hcluster.com, k = k, border = "red")
hc.avg = cutree(hcluster.avg,k)
Rand(hc.avg, cancer.lab)
#----------------------------
#single linkage
hcluster.sing = hclust( dist(cancer.data) , method = "single")
#dev.new()
plot( hcluster.sing, xlab = "", ylab = "", main = "Single Linkage", cex = 0.4)
rect.hclust(hcluster.com, k = k, border = "red")
hc.sing = cutree(hcluster.sing, k)
Rand(hc.sing, cancer.lab)

#unique(cancer.lab)

#dev.new()
#plot( pc.cancer$x[,1], pc.cancer$x[,2],xlab ="1st Principal Comp.", ylab = "2nd
    Principal Comp.",main = "Complete Linkage",col = hc.com+5,cex = 1.5)
#text( pc.cancer$x[,1], pc.cancer$x[,2],labels = colors,cex = 0.7 )
#----------------------------
#9.Kmeans
```

```
set.seed(15)
km.out  = kmeans(cancer.data, k)
Rand(km.out$cluster,cancer.lab)
#dev.new()
#plot( pc.cancer$x[,1], pc.cancer$x[,2],xlab = "1st Principal Comp.", ylab = " 2nd
    Principal Comp.", col = km.out$cluster,cex = 1.5)
#text( pc.cancer$x[,1], pc.cancer$x[,2],labels = colors,cex = 0.7 )
#table(hc.com,km.out$cluster)
#Rand(hc.com, km.out$cluster)
#A = c(1,1,2,3)
#B = c(1,1,1,2)
#Rand(A,B)
#-----------------------------
#10 . Convex Clustering

datamat = as.matrix(cancer.data)
carp.out = CARP(datamat)
names(carp.out)
convex.clust.lab = get_cluster_labels(carp.out, k = k)
Rand(convex.clust.lab,cancer.lab)
#dev.new()
#plot( pc.cancer$x[,1], pc.cancer$x[,2],xlab = "1st. Principal Comp." , ylab = "2nd
    Principal Comp.", col = convex.clust.lab,cex = 1.5)
#text( pc.cancer$x[,1], pc.cancer$x[,2],labels = colors,cex = 0.7 )
#-------------------------------
#11. sparse hierarchical clustering

#choose the tuning parameter
#gene_variances = apply(cancer.data, 2, var)
#top_genes = order(gene_variances, decreasing = TRUE)[1:1000]
#cancer.data.filtered = cancer.data[, top_genes]
cancer.data3 = as.matrix(cancer.data)
perm.out = HierarchicalSparseCluster.permute(cancer.data3,wbounds = NULL,nperms = 15)
#print(perm.out)
plot(perm.out)
perm.out$bestw
sparhclust.out = HierarchicalSparseCluster(cancer.data3, method ="complete", wbound  =
    perm.out$bestw, niter = 15)
hcspar.out = cutree(sparhclust.out$hc, k)
Rand(hcspar.out , cancer.lab)
#plot( pc.cancer$x[,1], pc.cancer$x[,2],xlab = "1st. Principal Comp." , ylab = "2nd
    Principal Comp.", col = hcspar.out,cex = 1.5)
#text( pc.cancer$x[,1], pc.cancer$x[,2],labels = colors,cex = 0.7 )
#---------------------------------
#12. MADD
madd.out = gMADD(1,1,k,15, cancer.data3)
Rand(madd.out,cancer.lab)
#plot( pc.cancer$x[,1], pc.cancer$x[,2],xlab = "1st. Principal Comp." , ylab = "2nd
    Principal Comp.", col = madd.out+3,cex = 1.5)
#text( pc.cancer$x[,1], pc.cancer$x[,2],labels = colors,cex = 0.7 )

#Rand Indices
Rand(hc.com,cancer.lab)
Rand(hc.avg, cancer.lab)
Rand(hc.sing, cancer.lab)
Rand(as.numeric(convex.clust.lab),cancer.lab)
Rand(km.out$cluster,cancer.lab)
Rand(hcspar.out , cancer.lab)
Rand(madd.out, cancer.lab)
#14.tsne Plots

#t-SNE
tsne_out <- Rtsne(cancer.data, dims = 2, perplexity = 15)
#Complete Hi. Clustering
plot(tsne_out$Y[,1], tsne_out$Y[,2], col = hc.com+2,pch = 19, xlab = "t-SNE 1",
     ylab = "t-SNE 2",main = "t-SNE Plot of Cancer Hi. Clusters with Complete Linkage")
text( tsne_out$Y[,1], tsne_out$Y[,2],labels = colors,cex = 0.7 )
#K-Means
plot(tsne_out$Y[,1], tsne_out$Y[,2], col = km.out$cluster+3,pch = 19, xlab = "t-SNE 1",
     ylab = "t-SNE 2",main = "t-SNE Plot of Cancer  Clusters with K-Means")
text( tsne_out$Y[,1], tsne_out$Y[,2],labels = colors,cex = 0.7 )
#Convex

```

```r
plot(tsne_out$Y[,1], tsne_out$Y[,2], col = as.numeric(convex.clust.lab)+2,pch = 19, xlab
     = "t-SNE 1",
     ylab = "t-SNE 2",main = "t-SNE Plot of Cancer Clusters with Convex Clustering")
text( tsne_out$Y[,1], tsne_out$Y[,2],labels = colors,cex = 0.7 )

#Sparse Hi.

plot(tsne_out$Y[,1], tsne_out$Y[,2], col = hcspar.out+2,pch = 19, xlab = "t-SNE 1",
     ylab = "t-SNE 2",main = "t-SNE Plot of Cancer Sparse Hi. Clusters with Complete
         Linkage")
text( tsne_out$Y[,1], tsne_out$Y[,2],labels = colors,cex = 0.7 )

#MADD- KMeans

plot(tsne_out$Y[,1], tsne_out$Y[,2], col = madd.out+4,pch = 19, xlab = "t-SNE 1",
     ylab = "t-SNE 2",main = "t-SNE Plot of Cancer Clusters with K-Means with MADD
         distance")
text( tsne_out$Y[,1], tsne_out$Y[,2],labels = colors,cex = 0.7 )
```

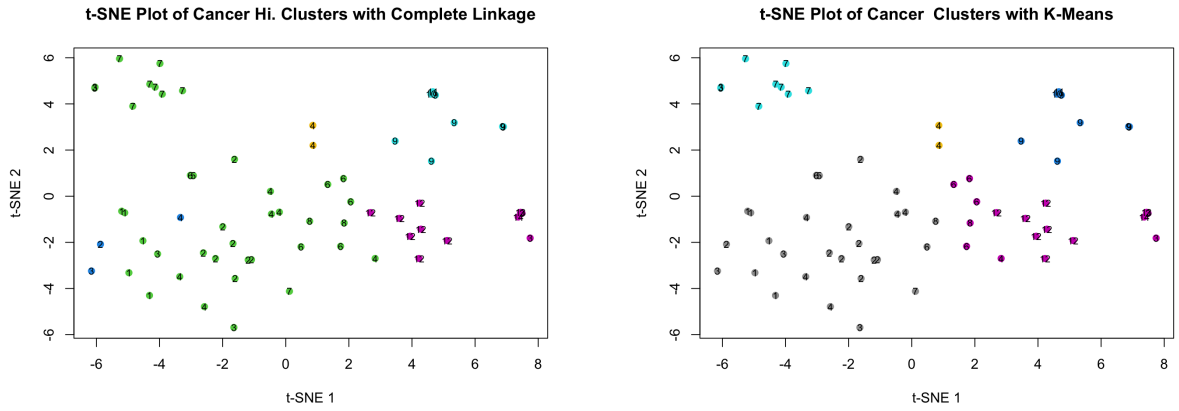
6.3 Interpretation

6.3.1 Plots

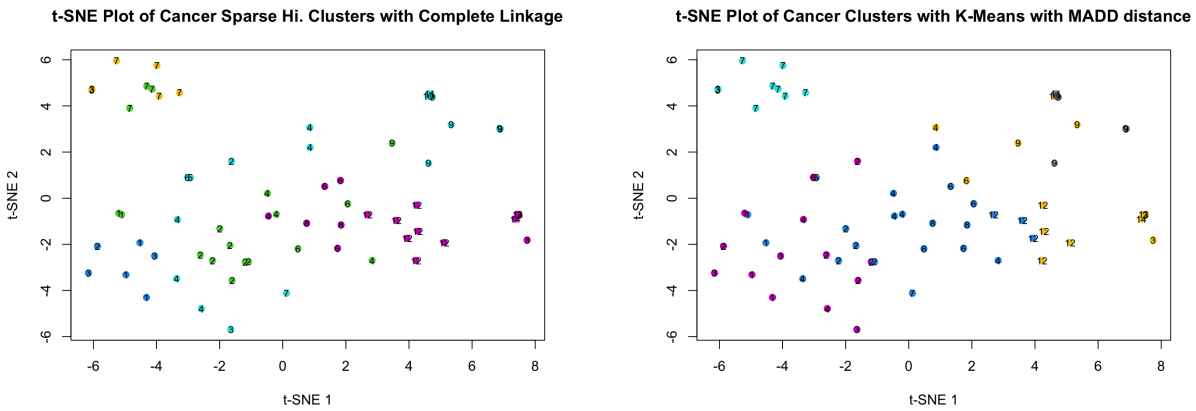


(a) Complete Linkage (b) Average Linkage (c) Single Linkage

Figure 1: Comparison of Hierarchical Clustering using Complete, Average, and Single Linkages.



(a) Complete Linkage H-Clust (b) K-Means Clustering



(a) Sparse Hierarchical Clustering

(b) K-Means with MADD

Figure 3: Comparison of four different clustering algorithms

6.3.2 Rand Index

As we have defined the rand index, the perfect clustering has Rand index 0 , In othe rwords, lower the rand index, better is the clustering. Thus we find that the Rand indices are as following :

Table 1: Rand Indices of Different Clustering Algorithms for dividing into 5 Clusters

	Rand Index
Complete Linkage Hi. Clust.	0.3809524
Average Linkage Hi. Clust.	0.6483135
Single Linkage Hi. Clust.	0.7237103
Convex Clustering	0.4384921
K-means	0.281746
Sparse Hi. Clust.	0.2311508
K-means with MADD	0.2385913

6.3.3 Interpretation

1. In the unsupervised problem where the data-set has to be divided into 5 clusters we can see from the table above that Convex Clustering has not performed very well but Hierarchial with complete linkage has out-performed the other linkage methods, and also, the Sparse and HDLS specific modifications have clearly outperformed the classical clustering techniques.
2. If we describe the clustering qualitatively, then we can see that from the t-SNE plots, that
    i. **Complete Linkage H-Clust**
    - **Verdict:** Excellent at isolating extreme outliers, but struggles with the central mass.
    - **Successes:** It perfectly isolates the distinct biological islands at the extremes. The entire Melanoma group (7) is perfectly captured in the top left (magenta), and the Leukemia/K562 family (9, 10, 11) is cleanly captured at the bottom (black).

- **Failures:** It lumps the Breast cancer family (3, 13, 14) and the Colon cancer lines (12) together into a single massive cluster (blue) in the center, failing to recognize the clear spatial gap between them.

ii. **K-Means Clustering**

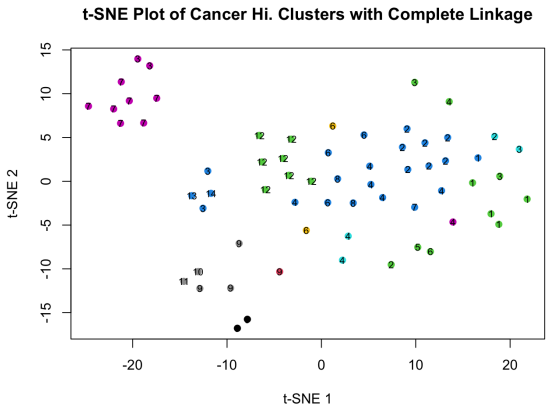
- **Verdict:** Exhibits classic failure modes due to its rigid spherical assumptions.
- **Failures:** Standard K-Means performs very poorly here. Because t-SNE reveals elongated, non-spherical structures, K-Means arbitrarily fractures clear biological groups. It slices the tight Melanoma island (7) into three different clusters (cyan, black, gold) simply because it spans a wide horizontal area. It similarly chops up the Leukemia group at the bottom into multiple distinct clusters.

iii. **Sparse Hi. Clusters with Complete Linkage**

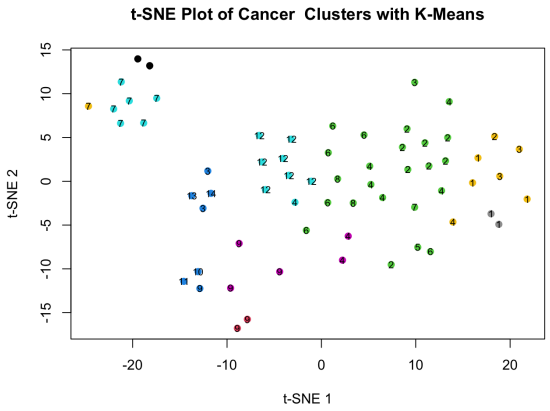
- **Verdict:** The strongest overall performer for a 5-cluster division on this dataset.
- **Successes:** By using feature selection to drop noisy genes, the clustering becomes highly logical. It perfectly isolates the Melanoma (7, magenta) and Leukemia families (9, 10, 11, green). It also creates a very coherent central cluster (black) that smartly groups the Breast family (3, 13, 14) with the Colon lines (12), leaving the more ambiguous, intermingled cell lines to be cleanly split into the remaining two clusters on the right.

iv. **K-Means with MADD Distance**

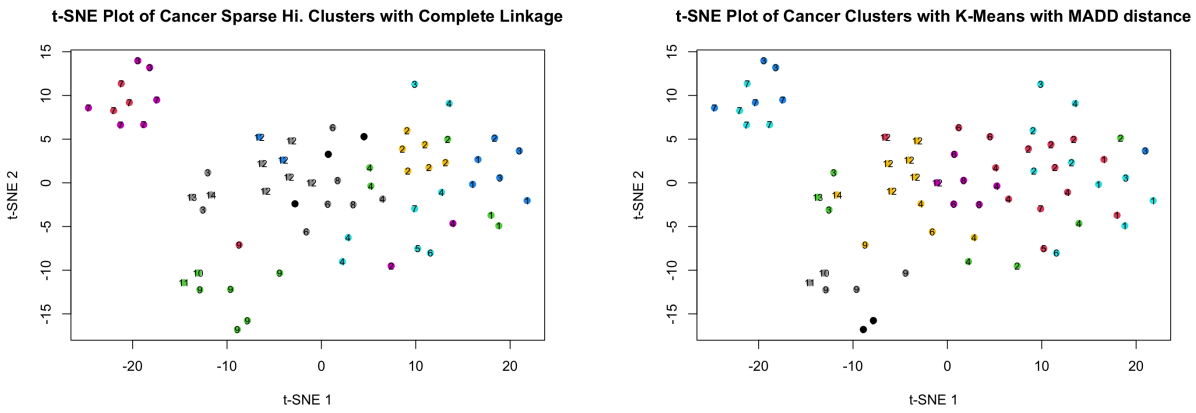
- **Verdict:** A massive improvement over standard K-Means, successfully overcoming spherical limitations.
- **Successes:** Using the Mean Absolute Deviation of Distances allows the algorithm to respect the natural density gaps. Unlike standard K-Means, it successfully keeps the Melanoma group (7, blue) and Leukemia group (9, 10, 11, black) completely intact.
- **Performance:** It also does an excellent job separating the Breast family (3, 13, 14, green) from the Colon group (12, gold), effectively mapping the four most visually distinct geometric regions into four distinct clusters.



(a) Complete Linkage H-Clust



(b) K-Means Clustering



(a) Sparse Hierarchical Clustering

(b) K-Means with MADD

Figure 5: Comparison of four different clustering algorithms

3. If we want to set $K = 14$ and see which algorithm performs the best in recovering the true clustering structure, we can see that the Rand indices of Hierarchical, K-Means, Sparse Hierarchical and MADD-K-Means Clustering are respectively: 0.1448413, 0.1661706, 0.1344246, 0.1527778, So, we can see the HDLS modifications of the classical clustering algorithms are performing better.

4. a) **Hierarchical Clustering with Complete Linkage**
   - **Verdict:** Best at isolating distinct biological families.
   - **Successes:** It perfectly captured the Melanoma group (7), successfully grouped the central Colon mass (12), and largely grouped the Breast/MCF7 family (3, 13, 14) together.
   - **Failures:** It completely failed to differentiate the massive, noisy cluster on the right side, assigning almost all the Renal, Ovarian, and NSCLC cell lines to a single massive "catch-all" cluster.

   b) **K-Means**
   - **Verdict:** Severely penalized by its spherical assumption.
   - **Successes:** It managed to identify the core of the Colon (12) group.
   - **Failures:** Because t-SNE reveals elongated and irregular manifolds, K-Means arbitrarily slices distinct biological groups in half. It splits the tight Melanoma group (7) simply because the points are spread out horizontally, and fractures the Leukemia/K562 group (9, 10, 11) into three different clusters.

   c) **Sparse Hierarchical Clustering (Complete Linkage)**
   - **Verdict:** Feature selection slightly destabilized the distinct groups.
   - **Performance:** In this specific projection, it appears slightly less cohesive than standard Complete Linkage. While it keeps the Melanoma (7) and Breast (3, 13, 14) groups relatively pure, it heavily fractures the central mass and splits the Leukemia group (9, 10, 11).

   d) **K-Means with MADD Distance**
   - **Verdict:** Marginal improvement over standard K-Means, but still constrained.
   - **Performance:** It performs slightly better than standard K-Means, keeping the Melanoma (7) group slightly more cohesive and doing a better job grouping the Colon (12) cell lines. However, it still struggles with the irregularly shaped, elongated mass on the right side, chopping it up into arbitrary clusters that do not map cleanly to the true labels (1, 2, 4).

7 Acknowledgements

I would like to thank Dr. Shyamal Krishna De, Applied Statistics Unit(ASU), ISI Kolkataand Dr. Saptarshi Chakraborty, University of Michigan for their guidance and valuable suggestions.

8 References

1. Elements in Statistical Learning, T. Hastie, R. Tibshirani, J. Friedman
2. Introduction to Statistical Learning, T.Hastie, R. Tibshirani, D. Witten, G. James
3. Statistical Learning with Sparsity, T. Hastie, R. Tibshirani, M. Wainwright
4. The Why and How of Convex Clustering, E. Chi, A.Molstad, Z. Gao, J.Chi
5. Biconvex Clustering S.Chakraborty, J.Xu
6. Sparse Convex Clustering, B.Wang, Y.Zhang, W. Sun, Y.Fang
7. A framework for feature selection in clustering, R. Tibshirani, D. Witten
8. On Perfect Clustering of High Dimension,Low Sample Size Data, S.Sarkar and A.K.Ghosh
9. Some clustering based exact distribution-free k-sample tests applicable to high dimension, low sample size data, B.Paul, S.K.De, A.K.Ghosh